

Response to Reviewers

A PROBABILISTIC CHOREOGRAPHY LANGUAGE FOR PRISM

Marco Carbone and Adele Veschetti

General Response

We thank the Editor and the Reviewers for their detailed and constructive feedback. We have revised the paper accordingly and believe that the changes substantially improve both clarity and technical depth. Below we summarise the major changes and the conventions used in this response.

Major structural change: projection (Section 5). The definition of projection has been completely rewritten. In the previous version, the projection relied on explicit counters (natural numbers, denoted by ι) to generate fresh identifiers. This mechanism was cumbersome and could be error-prone. We now adopt a more direct approach by annotating choreographies with unique labels, which the projection can use directly. This yields a cleaner and more easily readable projection in the theoretical development. For practical reasons, our implementation still uses counters: these can be seen as a concrete realisation of the label-generation mechanism that the paper now abstracts into the choreography annotations. Because Section 5 was rewritten end-to-end, its changes are *not* highlighted in blue; we kindly ask the reviewers to read it over again.

Convention for this response. We have updated this response to make it more clear to the reviewers what changes have been made with respect to the original submission. Each reviewer comment is tagged with a unique identifier ($[R x.y]$, where x is the reviewer number and y is the sequential position of the comment). Comments are quoted literally from the original reviews so that they can be immediately recognised. Where the paper text was modified in response to a comment, the corresponding change is highlighted **in blue** in the accompanying highlighted PDF, and the same $[R x.y]$ tag appears in the margin next to the modified passage. When a single passage in the paper addresses comments from different reviewers, the margin tag lists both identifiers (e.g., $[R2.18/R3.3]$). Again, Section 5 was completely rewritten, so we did not highlight any changes there. Minor comments (typos, spelling inconsistencies, formatting) have been addressed throughout but are not individually highlighted.

Reviewer 1

Reviewer comment. [R1.1]

I found the technical result establishing operational equivalence between choreographies and their projections rather weak. The result shows that the initial step is in correspondence provided that all projections are synchronized on some general state ι . However, it seems to me that this synchronization is not preserved by reduction, i.e., after a reduction step some projections remain in an ι state while others do not. This may ultimately be irrelevant, but I believe the issue should be discussed in the paper. I would also expect a result establishing operational correspondence for a sequence of reductions.

Response. We thank the reviewer for highlighting the mismatch between standard operational-correspondence results for choreographies and this paper focussing on PRISM. PRISM modules are declarative (a fixed set of commands whose enabling depends on the current state), whereas choreographies have an explicit control-flow syntax; therefore, in the projection, control-flow is reified into the (extended) runtime state via the variables s_p , yielding an extended state space S_+ . As a consequence, reductions of the choreography correspond to transitions on S_+ in the projected model rather than to syntactic evolution of PRISM modules. In the revised manuscript, we make this design choice explicit and discuss the resulting asymmetry (projection-after-reduction may yield fewer commands than reduction-after-projection). At the end of Section 5, we include a concrete example illustrating that, while the set of commands may differ, the projected models exhibit the same observable behaviour up to a suitable PRISM-induced equivalence; we leave the identification of the most appropriate equivalence (e.g., probabilistic bisimulation) and the corresponding strengthening of the theorem to future work.

Changes in the manuscript. We expanded the discussion around Theorem 1 to clarify (i) why standard reduction-to-reduction correspondence does not directly apply to PRISM, (ii) how control-flow information is encoded in the extended state S_+ via the distributed state variables, and (iii) why the correspondence statement is formulated in terms of transitions on S_+ rather than syntactic changes of modules. We also added a detailed remark and worked example to make the asymmetry and its implications explicit, and we added a paragraph outlining how a future extension based on probabilistic bisimulation could capture correspondence across different command sets.

Reviewer comment. [R1.2]

Furthermore, the proof of this correspondence (Theorem 1) reads more like a proof sketch than a complete proof; a more detailed and carefully paced proof would be preferable.

Response. We thank the reviewer for this suggestion. In the revised version, we have expanded the proof of Theorem 1 and made its structure more explicit, in particular by spelling out the interaction case in detail and by making clear the key invariants on the extended state S_+ used in the correspondence argument. To keep the exposition readable, we do not enumerate every PRISM semantic step.

Changes in the manuscript. Proof of Theorem 1 rewritten and expanded and interaction case detailed.

Reviewer comment. [R1.3]

I have also some concerns about the way in which the choreography state is projected on different participants. I think that the presentation will benefit from a careful treatment of this aspect.

Changes in the manuscript. Definitions 4 (CTMC) and 5 (DTMC) have been updated. Now states are pre-generated making it more clear how the projection works.

Reviewer comment. [R1.4]

When giving examples, the use of prime variables is not clear at this point.

Response. We added a clarifying sentence in the motivating example paragraph of Section 1, right after the first use of x' and y' . The sentence is highlighted in blue and tagged [R1.4] in the margin.

Changes in the manuscript. Added the following sentence (highlighted in blue, tagged [R1.4]) in Section 1, after the description of the two branches:

Here, we adopt PRISM notation where the prime symbol, e.g., x' , y' , indicates the next-state value of the corresponding variable, that is, the value it will take after the transition occurs.

Reviewer comment. [R1.5]

I did not understand the explanation of foreach. What is the role of k in such an expression? What is op ? A formal encoding would be welcome.

Response. We apologise for the limited explanation of this construct. The basic idea is to provide an operator that quantifies over indices via the predicate $op(i)$. The formal translation is straightforward: we add one update for each role, assuming that the relevant roles can be determined statically.

Changes in the manuscript. We have updated the text where the foreach construct is discussed (Section 3.3), with a more precise explanation of what it does. The new text is highlighted in blue and tagged [R1.5] in the margin.

Reviewer comment. [R1.6]

I am puzzled about the notion of Non-deterministic Synchronization. The encoding shown for the example looks quite deterministic to me. Moreover, as written, it seems that you are lifting features of the implementation language (PRISM) to the choreography level. The encoding then arbitrarily introduces a synchronization. I have a couple of questions here:

1. Does the specification determine the direction of communication? In the provided example, it seems that the interaction could also be expressed in the opposite direction, i.e., $q \rightarrow \{p\}$. Is the direction of the interaction relevant, or is it never relevant?
2. Does any synchronization specification yield a correct choreography? Should there not be a requirement for some well-formedness condition on the allsync constructor?
3. Could you provide a formal definition of the encoding?

Response. The terminology we used is incorrect, thanks for pointing this out. `allsynch` is not "nondeterministic" but it is still a probabilistic choice. We have rewritten (and expanded) the entire paragraph with a clearer explanation. As for the detailed questions:

1. in our choreography language, writing $q \longrightarrow \{p\}$ or $p \longrightarrow \{q\}$ is the same. However, it has an impact on the projection, since we need to distinguish one of the modules (the one on the left) because it has a different projection.
2. The notion of strong connectedness can be extended to `allsynch`, as it is, after all, an interaction between the modules p_i . However, this is not necessary if we first translate it into standard interactions (and if-then-else constructs), since strong connectedness can then be checked at that level. In the new example we have provided, strong connectedness does not hold because the nested if-then-else constructs are not connected. As this part is not a formal development but rather a section justifying how our implementation extends the theoretical framework, we do not consider it necessary to present a connected choreography here. In any case (as noted in a comment added to the projection section), these choreographies can be transformed into connected ones by introducing additional interactions.
3. the translation should be straightforward once there is a precisely defined language for guards. See new text added.

Changes in the manuscript. The paragraph has been rewritten. The new text is highlighted in blue and tagged [R1.6] in the margin of Section 3.3.

Reviewer comment. [R1.7]

Example 3: The modules in this example share variables (y and x). I wonder whether there is a choreography that describes their behavior, since my understanding is that variables are local to modules in choreographies.

Response. In the example, x and y are indeed owned by two different modules and are therefore shared only in read mode. In PRISM, all modules can read all variables, but only the owning module can modify its own variable. This is consistent with our choreographic reading, where updates are only performed as part of an interaction, so the write is still attributed to a specific role in that interaction. The only caveat is that, in choreographies, guards in conditionals must be local (i.e., evaluated by a single role). Thus, for this example, we could simulate the interactive execution (sync on a) by nesting two if-then-else constructs. However, since choreographies focus only on interactions, the internal actions (no label) cannot be simulated.

Reviewer comment. [R1.8]

The choice of annotating choreographies with labels seems somewhat ad hoc. Why are they not introduced as a syntactic element of choreographies (as is often done with communication channels in standard choreographies), given that they are necessary and play a role in the implementation?

Response. It is certainly true that labels are a key aspect of the implementation. However, the purpose of this work is to define a higher-level language that abstracts away from such implementation details. Therefore, labels are part of the projection algorithm and are irrelevant to the programmer who writes a choreography. Nevertheless, we now use an intermediate choreographic language and a function for annotating choreographies in a "safe" way.

Reviewer comment. [R1.9]

Projection of conditionals: I do not understand why the conditional is not projected. I would expect that E should be evaluated locally by p (hence all relevant information must be local to p). This contrasts with the statement on page 5: “The term $\text{if } E@p \text{ then } C1 \text{ else } C2$ denotes a system where module p evaluates the guard E (which can contain variables located at other modules) and then (deterministically) branches accordingly.” I presume that there is some underlying assumption on the projection, but I could not find its definition.

Response. We believe this is a misunderstanding, and we are grateful for the opportunity to clarify it. The conditional *is* projected, but only in the module of the evaluating role p . This is consistent with the intended semantics: the guard E is evaluated by p and the result deterministically selects the branch. The point that may be confusing is that we do *not* require E to be local to p . As stated on page 5, E may mention variables owned by other modules; in PRISM, all modules can read all variables, while only the owning module can modify them. Therefore, the projection places the conditional in p ’s module, and the guard can still refer to remote variables, exactly as in the PRISM syntax.

Reviewer comment. [R1.10]

Projection of choreography variable X : This relies on $\text{defs}(X)$. I presume that this function should be explicitly included in the definition. Moreover, I assume that you need to impose some conditions, such as requiring that the image of any $X.i$ is distinct from $\text{nodes}(C)$ and also distinct from the nodes in all definitions. The passage on page 13 (“For recursive calls. . . defs ”) appears somewhat rushed, and I believe defs deserves a formal definition.

Response. We agree with the reviewer.

Changes in the manuscript. In the revised formulation, counters are replaced by explicit labels on choreographies, which makes the treatment of variables cleaner and more uniform. We have also expanded the proof for this case and included more details to make the reasoning fully explicit.

Reviewer comment. [R1.11]

Projection of updates: Are you assuming that updates at the choreography level are well-formed in some way? For instance, could an assignment $y' = x$ occur where y belongs to p and x to q ? If not, this restriction should be stated clearly. Moreover, from which element of the choreography is the assignment derived? If I have missed this, it should nonetheless be clarified explicitly in the paper.

Response. As discussed above, assignments such as $y' = x$ are permitted even when x is located at p and y is located at q . At the choreography level, the restriction is simply that, in an interaction $p \rightarrow \{p_1, \dots, p_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j$, every update in each u_j assigns to a variable owned by one of $p, p_1, \dots, p_n$.

Changes in the manuscript. We have added Assumption 1 in Section 3, highlighted in blue and tagged [R1.11] in the margin.

Reviewer comment. [R1.12]

Definition 4: Is it not the case that the choreography is annotated? Hence, the interaction $p \rightarrow \{p_1, \dots, p_n\}$ should also be labeled. Furthermore, from where are the labels l_j drawn? Does this mean that every interaction uses the same set of labels $l_1, \dots$?

Response. Interactions are now also annotated. See new version.

Reviewer comment. [R1.13]

How are variables quantified? Are S , S' , and ι universally quantified?

Response. Yes, although now ι has disappeared.

Reviewer comment. [R1.14]

The correspondence theorem (Theorem 1) appears somewhat weak, since it merely shows that projections can mimic the first computational step, as far as the state is concerned. It does not provide evidence that subsequent steps preserve equivalence. I am particularly concerned that the projections may become misaligned with respect to ι . Thus, it is not the case that the second step necessarily advances all projections corresponding to a given ι . This issue is clear in the case of conditionals, and I also wonder about process variables.

Response. See our reply about your first comment.

Reviewer comment. [R1.15]

Although I can follow the reasoning, I wonder what the fundamental limitation is here. Is it because the choreographies are purely probabilistic, without non-deterministic choices? Would the dining cryptographers require some form of non-determinism?

Changes in the manuscript. The explanation is in the Dining Cryptographers paragraph of Section 6, highlighted in blue and tagged [R1.15] in the margin.

Reviewer 2

Reviewer comment. [R2.1]

The main changes as stated by the authors at the end of the Introduction are:

- Previously omitted details in definitions and proofs. I consider this a minor extension. The paper does explain which specific definitions, details or proofs have been included. Neither does the cover letter.
- A thorough analysis of richer language constructs incorporated in the implementation. The Introduction does not explain which specific analysis and constructs they refer to. The paper does not specify in which section this is to be found. In principle, this sounds like a significant extension but with the paper does not help the reader in identifying it (one would need to read both papers and guess). Also here, the cover letter does not provide additional details.
- New use cases analysed with the proposed approach. In consider this as a significant extension, and aligned with the recommendation of the COORDINATION reviews. As in the above case, the paper will be more clear if specific pointers to sections would be provided. As it is now, distinguishing the new from the old cases is left as a homework to the reader. My guess is that these are the Peer-To-Peer protocol, the leader election protocol and the dining cryptographers.

[Reviewer’s clarification in the second round:] “The paragraph ‘Changes with respect to the conference version’ does not use references to specific sections where the novel contributions are to be found. You should add such references. Moreover, the examples of the newly introduced additional language constructs must be listed in the text.”

Response. We thank the reviewer for the clarification, and we apologise: in our previous reply we did not address what was actually being asked. We have now revised the “Changes with respect to the conference version” paragraph at the end of the Introduction to include explicit section references for each change. We kept the edit minimal: the original prose is preserved and only the section pointers were added inline (highlighted in blue); the paragraph is tagged [R2.1] in the margin. The explicit naming of the three additional language constructs is addressed in the “optional expansion” at the end of this item.

Changes in the manuscript. In the paragraph “Changes with respect to the conference version” at the end of Section 1, we inserted the following explicit section pointers (each shown in blue in the highlighted PDF):

- “...complete proof of the main theorem (**Section 5**), which were previously omitted...”
- “...revised projection function (**Section 5**) that provides a cleaner formalisation...”
- “...additional language constructs (**Section 3.3**), which are also supported by the implementation...”
- “...expand the related work section (**Section 7**)...”
- “...new use cases analysed through choreographies (**Section 6**)...”

Optional, more verbose expansion. The reviewer also asked that the three new language constructs be listed by name in the text. We deliberately did *not* inline the construct names in the current revision because we felt it would make the paragraph cumbersome for an introduction. If the reviewer prefers a fully itemised and more explicit version of this paragraph, we are happy to replace the current one with the following expanded version, which also lists the new constructs explicitly:

This article is an expanded version of our paper presented at COORDINATION 2024. With respect to that version, the main differences are the following.

- *Complete proof of the main theorem (Section 5).* Theorem 1, which was given only as a sketch in the conference version, is now accompanied by a full proof, including a detailed treatment of the interaction case.
- *Revised projection function (Section 5).* The projection no longer relies on natural-number counters for the control state. Instead, choreographies are first decorated with explicit control and state labels through the annotation function `annot` (Definition 3); the projection is then defined directly on annotated choreographies, yielding a cleaner formalisation.
- *Additional language constructs (Section 3.3).* A new subsection introduces three constructs that are syntactic sugar for the core language and are supported by our implementation: (i) parametric modules $p[i]$, which allow indexing modules over a finite set; (ii) the `foreach` construct, of the form `foreach ( $k$  op  $i$ )  $u @ A[k]$` , which iterates an update over a set of indexed modules; and (iii) the `allsynch` construct, which expresses a synchronisation in which several modules independently choose a branch among multiple alternatives.
- *New case studies (Section 6).* The experimental evaluation includes three new case studies: the Simple Peer-to-Peer Protocol, the Synchronous Leader Election Protocol, and the Dining Cryptographers Protocol. We also added a table comparing the size of choreographies with that of the corresponding generated PRISM models.
- *Extended Related Work and Discussion (Section 7).* The Related Work part has been extended with a new paragraph relating our setting to the implementability/realisability literature. The Discussion part has been expanded with a more precise characterisation of the class of systems amenable to our framework, and with a new discussion of the trade-offs of choreographic programming with respect to process-oriented programming.

Reviewer comment. [R2.2]

For the COORDINATION paper the following remark was done in the reviews: “The introduction should be more clear about which kind of concurrent process is amenable for choreographies, and for which class of concurrent processes choreographies are not suitable.” The comment still applies for the new introduction.

Response. We added a dedicated paragraph in Section 1 that makes explicit which kind of concurrent systems our approach targets and which kinds of systems are less suited to it. The new paragraph is highlighted in blue and tagged [R2.2] in the margin of the revised PDF.

Changes in the manuscript. The following paragraph was added to Section 1, just before the presentation of the language (highlighted in blue and tagged [R2.2] in the margin):

Choreographic programming naturally applies to distributed systems whose behaviour can be described through structured interaction protocols. Systems whose dynamics rely heavily on loosely coordinated, highly unstructured, or unpredictable communication patterns are generally less amenable to this approach. Within its intended scope, however, choreographic programming has proved effective in practice, since choreographies enable the automatic generation of decentralised implementations that are correct by construction.

Reviewer comment. [R2.3]

The experimental analysis in Section 7 needs to be improved. What I miss the most is an overview of the results, with some measurable criteria.

Response. Section 6 (previously Section 7, now Section 6 after the removal of the old Section 6 – see [R2.4]) has been extended with a new opening paragraph and a new “Code Size Comparison” subsection containing Table 1. Both are highlighted in blue and tagged [R2.3] in the margin. The criterion of equivalence used in the evaluation is clarified under [R2.18/R3.3].

Reviewer comment. [R2.4]

Section 6 is irrelevant and can be removed.

Changes in the manuscript. Removed. The relevant information (Java/ANTLR implementation) was moved into the opening paragraph of Section 6 (tagged [R2.3/R2.4]). As a consequence, all subsequent section numbers shifted down by one.

Reviewer comment. [R2.5]

It is confusing that you use $\lambda_{1,2}$ as label in page 2 (encoded then as a an b), and later in page 3 λ is the rate of the CTMC actions. I recommend trying to make the use of symbols more coherent.

Changes in the manuscript. The motivating example paragraph in Section 1 was rewritten so that λ_1, λ_2 are consistently presented as *rates* and the labels a, b are clearly separated as PRISM command labels. The new text is highlighted in blue and tagged [R2.5] in the margin.

Reviewer comment. [R2.6]

Relate to this: the text uses sometimes the math symbols and sometimes the full name of the greek letters λ , μ , θ . If you refer to code snippets, where the latter is used, I would use that in the main text too to ease readability. Note that you refer to other code symbols using the code format (e.g. `CheckOutState`).

Changes in the manuscript. In Section 2, λ, μ, θ replaced by code-formatted `lambda`, `mu`, `theta` (two locations, highlighted in blue, tagged [R2.6]).

Reviewer comment. [R2.7]

I don’t know if this intentional, but if one wants to specify no update, one has to write $x' = x$ (a void update). In PRISM one can simply write `true`, if I recall properly. This is related also to the syntax of PRISM introduced in 4.1.

Response. From a theoretical point of view, there is no difference, therefore we will leave this as it is. Our implementation allows also `True`.

Reviewer comment. [R2.8]

Are you sure Def. 1 works as intended? What happens if x appears in u ? For example, $[x' =$

$0, y' = x]$. In Prism, y will be updated to the value of x in S (see remark at the bottom of <https://www.prismmodelchecker.org/manual/ThePRISMLanguage/Commands>), but in your case it will be set to the new value of x (0). This is also relevant to the definition of μ in page 9 and needs to be fixed.

Response. Yes, our definition was incorrect.

Changes in the manuscript. Definition 1 in Section 3 has been rewritten to handle multiple updates simultaneously. The new definition is highlighted in blue and tagged [R2.8] in the margin.

Reviewer comment. [R2.12]

General remark: the way you define unique identifiers for the control points in the choreographies is not self-explainable. Readers familiar with building control flow graphs and similar graphical structures from code will get it, but other readers may find it cryptic. I would suggest enriching the explanation of how control points sq are correctly created.

Changes in the manuscript. Section 5 has been rewritten from scratch with a new label-based mechanism (Definitions 2 and 3) that replaces the old counter-based construction and makes the generation of control-point identifiers explicit. We kindly ask the reviewer to read Section 5 afresh.

Reviewer comment. [R2.13]

“The projected update $u_j \downarrow q$ acts as a filter on the list of updates in u_j ” – is this defined in the paper?

Changes in the manuscript. Section 5 has been rewritten from scratch. The explanation of $u_j \downarrow q$ now appears immediately after the first projection rule, together with Assumption 1 in Section 3 (see [R1.11]). We kindly ask the reviewer to read Section 5 afresh.

Reviewer comment. [R2.18]

“we demonstrate that PRISM behaves similarly” – be more precise about what you mean by “similarly”. Some notion of bisimulation for CTMCs/DTMCs? Equi-satisfaction of some subset of properties? You may want to try out STORM’s bisimulation support. It could give you higher confidence than just checking that the models behave similarly for some specific properties.

Response. We apologise for not having addressed this comment in our previous reply. We rephrased “behaves similarly” as “exhibits equivalent behaviour” and clarified that we mean equi-satisfaction on the benchmark properties (reachability, time-bounded probabilities, expected values), not bisimilarity. We did not use STORM’s bisimulation support because our comparison is property-based.

Changes in the manuscript. In Section 6, the introductory paragraph was rewritten: item (ii) now reads “PRISM exhibits equivalent behaviour...” followed by a new sentence defining the criterion. Highlighted in blue and tagged [R2.18/R3.3] in the margin.

Reviewer comment. [R2.19]

“Any discrepancies between the original and generated models are due to inherent variations in the simulation-based calculation of probability.” – have you tried not to use simulations but to use exact calculations? Perhaps also with rationals instead of floating points.

Response. We apologise for not having addressed this comment in our previous reply. For the four smaller benchmarks, PRISM performs exact model checking; simulation is used only for Bitcoin PoW and Hybrid Casper, where the state space is too large for exact verification. The quoted sentence refers specifically to those two cases. We have now made this distinction explicit in the paper.

Changes in the manuscript. Added a clarifying sentence in the preamble of Section 6 stating which benchmarks use exact model checking and which use simulation. Highlighted in blue and tagged [R2.19] in the margin.

Reviewer 3

Reviewer comment. [R3.1]

The pros and cons of choreographic programming as opposed to process-oriented programming are not elaborated upon very much. I understand that flexibility is sacrificed in favour of predictability with the advantage of guaranteeing correctness-by-construction. What is lost by doing so? And while the authors mention in the Introduction that their “approach can be used for a proper subset of distributed systems”, the size of this subset and type of systems populating it is not even hinted at. The authors only write that “Some of the case studies presented in the PRISM documentation [2] cannot be modeled” and then provide two concrete examples of what cannot be modelled. Could the authors state something more general?

Changes in the manuscript. We added a new paragraph in Section 7 (Discussion) characterising what is lost by adopting the choreographic discipline (highlighted in blue, tagged [R3.1]). A complementary paragraph in Section 1 characterises which systems our approach targets (tagged [R2.2]).

Reviewer comment. [R3.2]

How does this relate to the notions of implementability and realisability as studied for transition systems and (team) automata? In the following papers from the literature, a global transition system or (team) automaton is given and the main question is whether or not there exists a distributed set of local (cooperating, synchronously communicating) transition systems or (component) automata, whose composition is equivalent or bisimilar to the global one.

- I. Castellani, M. Mukund, and P.S. Thiagarajan, Synthesizing Distributed Transition Systems from Global Specifications. In: FSTTCS 1999. LNCS 1738, pp. 219–231. Springer, 1999.
- M. Mukund, From Global Specifications to Distributed Implementations. In: Synthesis and Control of Discrete Event Systems. Springer, 2002.
- M.H. ter Beek, R. Hennicker, and J. Proença, Realisability of Global Models of Interaction. In: ICTAC 2023. LNCS 14446, Springer, 2023, pp. 236–255.
- M.H. ter Beek, R. Hennicker, and J. Proença, Team Automata: Overview and Roadmap. In: COORDINATION 2024. LNCS 14676, Springer, 2024, pp. 161–198.

Changes in the manuscript. Added a new paragraph in Section 7 (Related Work) relating our approach to the implementability/realisability literature and citing the four suggested references. Highlighted in blue and tagged [R3.2] in the margin.

Reviewer comment. [R3.3]

Finally, during the experimental evaluation, the models generated by the authors’ approach are only compared to the original PRISM benchmarks with respect to certain properties and functionality, but not semantically. Could the authors imagine a more thorough comparison, e.g., whether or not the models are bisimilar, and why?

Response. We apologise that our previous reply pointed to “the end of Section 5”, which was misleading. The relevant discussion is in the introductory paragraph of Section 6, where we clarify that we compare models on quantitative properties, not bisimilarity. We do not aim for semantic equivalence at the model level.

Changes in the manuscript. See the paragraph tagged [R2.18/R3.3] in Section 6.